


# Laboratorio

Materia: Seminario de lenguajes Python

Carrera: Licenciatura en Sistemas

Docentes: Mg. Lic. María Alejandra Vranic

Lic. Gonzalo Cerbelli

Año cursada: 2026

# Índice

**1. Objetivo** **2**
A. Stack tecnológico obligatorio 2
**2. Metodología de trabajo en Github** **2**
**3. Consignas** **2**
A. ABM de Producto 2
B. ABM de Ventas 3
C. ABM de Carrito 5
D. Reglas de negocio 6

# 1. Objetivo

El objetivo de este laboratorio es aplicar los conocimientos adquiridos durante la cursada para el desarrollo de una API REST que permita gestionar ventas de productos. El enfoque estará puesto en la utilización del stack propuesto, buenas prácticas de desarrollo de código, manejo de datos y operaciones básicas (CRUD).

## A. Stack tecnológico obligatorio

- Python 3.12
- FastAPI
- SQLite
- SQLAlchemy
- Pandas
- Borb

# 2. Metodología de trabajo en Github

Deberán crear un repositorio en GitHub donde alojarán el código de la aplicación, el primer commit deberá ser el proyecto base (configuraciones necesarias para levantar la aplicación) y además deberán actualizar el archivo Readme (lo crean al momento de crear el repositorio) con el nombre del grupo, integrantes y funcionalidades realizadas por cada uno.

Cada nueva funcionalidad deberá ser trabajada en una branch distinta a la main, y mergearse mediante Pull Request, el nombre de las branchs con las funcionalidades nuevas tendrá que tener el siguiente formato:

**usuarioGitHub-nombreFuncionalidad.**

Por ejemplo con la funcionalidad del Login: **gonzalocerbelli-endpointCarrito**.

Al momento de *mergear* los cambios de la branch a main pueden utilizar la opción Squash and Merge (compacta todos los commits que tienen la branch en uno solo).

# 3. Consignas

## A. ABM de Producto

Se deberá implementar una API que permita gestionar productos, permitiendo crear un nuevo producto, obtener el listado de productos registrados en la tabla de base de datos, obtener un producto en particular, actualizar los datos de un producto y eliminar un producto. Para ello, los endpoints serán:

- POST /productos
- GET /productos
- GET /productos/{id}
- PUT /productos/{id}
- DELETE /productos/{id}

Los productos deberán contar con los siguientes datos:

- id (int pk autoincrement not null)
- nombre (str not null)
- precio (float not null)

*Ejemplo de cuerpo de endpoint POST /producto:*

```
{
 "nombre": "Esponja",
 "precio": "11.11"
}
```

*Ejemplo de respuesta de endpoint GET /producto/1:*

```
{
 "id": 1,
 "nombre": "Esponja",
 "precio": "11.11"
}
```

## B. ABM de Ventas

Se deberán implementar nuevos endpoints para gestionar ventas, permitiendo crear una nueva venta, obtener el listado de ventas registrados en la tabla de base de datos, obtener una venta en particular, actualizar los datos de una venta y eliminar una venta. Para ello, los endpoints serán:

- POST /ventas
- GET /ventas
- GET /ventas/{id}
- PUT /ventas/{id}
- DELETE /ventas/{id}
- POST /ventas/{id}/carrito

Las ventas deberán contar con los siguientes datos:

- id (int pk autoincrement not null)
- fecha (date not null)
- hora (time not null)
- id_carrito (fk int unique not null)

El precio total de la venta será calculado por la API al momento de obtener la información de una venta, sumando el precio de cada producto del carrito multiplicado por su cantidad.

Una venta tiene un único carrito asociado y un carrito puede pertenecer a una única venta (relación 1 a 1). El endpoint POST /ventas/{id}/carrito permite asignar un carrito a una venta o

reemplazar el ya asignado; si el carrito se encuentra asociado a otra venta, la API deberá devolver un error.

*Ejemplo de cuerpo de endpoint POST /ventas:*

```
{
 "fecha": "2000-01-01",
 "hora": "10:30",
 "id_carrito": 1
}
```

*Ejemplo de cuerpo de endpoint POST /ventas/1/carrito (asignar un carrito a la venta):*

```
{
 "id_carrito": 1
}
```

*Ejemplo de cuerpo de endpoint GET /ventas/1:*

```
{
 "id": 1,
 "fecha": "2000-01-01",
 "hora": "10:30",
 "precio_total": 4500,
 "carrito": {
     "id": 1,
     "fecha_creacion": "2000-01-02",
     "estado": "cerrado",
     "productos": [
           {
                 "id": 1,
                 "producto": {
                       "id": 2,
                       "nombre": "Arroz",
                       "precio": 300
                 },
                 "cantidad": 15
           }
     ]
 }
}
```

## C. ABM de Carrito

Se deberá implementar una nueva entidad denominada carrito de compras que permita agrupar productos con sus respectivas cantidades. Para ello, los endpoints serán:

- POST /carritos
- GET /carritos
- GET /carritos/{id}
- PUT /carritos/{id}
- DELETE /carritos/{id}
- POST /carritos/{id}/productos
- DELETE /carritos/{id}/productos/{id_producto}

Los carritos deberán contar con los siguientes datos:

- id (int pk autoincrement not null)
- fecha_creacion (date not null)
- estado (str not null) (por ejemplo: "abierto" o "cerrado")

La relación entre carritos y productos se realiza mediante la tabla carrito_producto, la cual deberá contar con los siguientes datos:

- id (int pk autoincrement not null)
- id_carrito (fk int not null)
- id_producto (fk int not null)
- cantidad (int not null)

De esta forma, un carrito contiene muchos productos (a través de los registros de carrito_producto) y un carrito puede ser asignado a una única venta (relación 1 a 1), mediante el endpoint POST /ventas/{id}/carrito.

El endpoint POST /carritos/{id}/productos recibe el id de un producto y una cantidad, y agrega ese producto al carrito. Si el producto ya se encontraba en el carrito, la cantidad recibida se suma a la cantidad existente.

El endpoint DELETE /carritos/{id}/productos/{id_producto} elimina el producto indicado del carrito. Solo se permite quitar productos de un carrito cuyo estado sea 'abierto'; si el producto no se encuentra en el carrito, se debe devolver un error.

*Ejemplo de cuerpo de endpoint POST /carritos:*

```
{
 "fecha_creacion": "2000-01-02",
 "estado": "abierto"
}
```

*Ejemplo de cuerpo de endpoint POST /carritos/1/productos (agregar un producto al carrito):*

```
{
 "id_producto": 2,
 "cantidad": 15
}
```

*Ejemplo de respuesta de endpoint GET /carritos/1:*

```
{
 "id": 1,
 "fecha_creacion": "2000-01-02",
 "estado": "abierto",
 "productos": [
    {
          "id": 1,
          "producto": {
                "id": 2,
                "nombre": "Arroz",
                "precio": 300
          },
          "cantidad": 15
    }
 ]
}
```

La eliminación de un carrito debe eliminar también sus registros asociados en carrito_producto. Si el carrito se encuentra asociado a una venta, la API deberá devolver un error y no se podrá eliminar.

*Ejemplo de endpoint DELETE /carritos/1/productos/2 (quitar un producto del carrito):* Este endpoint no recibe cuerpo; el id del producto se indica en la URL como "stringParameter".

## D. Reglas de negocio

Las reglas de negocio indican el comportamiento que deberá tener la API con respecto a los recursos (endpoints) y acciones (verbos) que se podrán realizar. La implementación deberá respetar las siguientes reglas de negocio:

**Ventas**

- El body de los endpoints POST /ventas y PUT /ventas/{id} nunca debe recibir el campo precio_total; su cálculo es responsabilidad de la API.
- Para registrar una venta o asignar/reemplazar el carrito mediante POST /ventas/{id}/carrito, el carrito indicado debe existir y no estar asociado a otra venta.

- Solo se permite asociar a una venta un carrito cuyo estado sea "cerrado".

**Carrito**

- Solo se pueden agregar o quitar productos de un carrito cuyo estado sea "abierto"; en caso contrario, los endpoints POST /carritos/{id}/productos y DELETE /carritos/{id}/productos/{id_producto} deben devolver un error.
- El id_producto recibido al agregar o quitar un producto debe referenciar a un producto existente y, en el caso del DELETE, encontrarse dentro del carrito; caso contrario se debe devolver un error.
- La cantidad recibida al agregar un producto debe ser mayor a 0; caso contrario se debe devolver un error.
- Si el producto ya se encuentra en el carrito, la cantidad recibida se suma a la existente (no se genera un registro duplicado).
- Un carrito que esté asociado a una venta no puede ser modificado (no admite PUT, ni agregar o quitar productos, ni volver a pasar a estado "abierto") ni eliminado; primero debe desvincularse mediante POST /ventas/{id}/carrito, y su estado debe permanecer "cerrado".

**Producto**

- No se debe permitir eliminar un producto que se encuentre registrado en algún carrito.